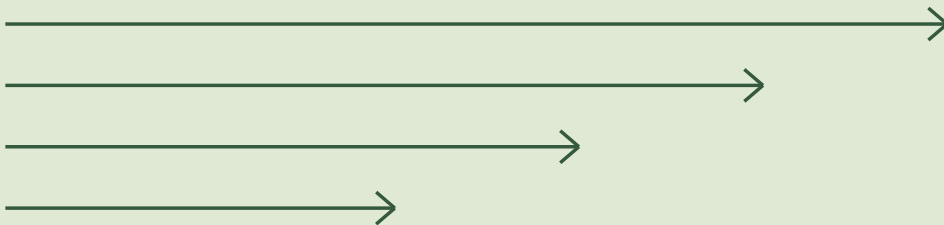




EVERY CAREER STAGE

The Fundamentals

Good answers start here.



12 QUESTIONS. CLEAR ANSWERS.

Follow-up prompts. A more confident next step.

Make the preparation count.

Read an answer, close the page, and explain the idea in your own words. Then work through the follow-up. A strong interview answer includes the conditions, trade-offs, and an example - not just a definition.

YOUR STUDY ROUTE

Foundations / 5 questions

Object-oriented design / 1 questions

Databases / 3 questions

Web essentials / 3 questions

Three passes through the kit

1. Understand: read each answer and identify the core idea.
2. Explain: answer aloud without looking and add an example.
3. Extend: solve the follow-up and write down what is still unclear.

About this edition

These questions are independently authored around common engineering interview topics. They are not sourced from or endorsed by an employer, and no interview outcome is guaranteed. Suggested answers are starting points: exact behavior can depend on your language, database, runtime, and business requirements. Validate implementation details against the official documentation for your stack.

01 / FOUNDATIONS**What does Big O tell you, and what does it leave out?**

Big O describes an upper bound on how resource use grows with input size. A linear scan is $O(n)$; binary search over sorted random-access data is $O(\log n)$. It hides constants and does not by itself predict wall-clock time. State whether you mean worst-case, average-case, or amortized cost, and include auxiliary space.

FOLLOW-UP

Compare scanning ten items with building a hash table for one lookup.

02 / FOUNDATIONS**When would you choose an array over a linked list?**

An array provides constant-time indexed access and usually good cache locality. A linked list supports constant-time insertion after an already-known node, but finding that node is linear and nodes add allocation overhead. Dynamic arrays occasionally resize; append is typically amortized $O(1)$. Choose based on the dominant operations, not the claim that linked-list insertion is always faster.

FOLLOW-UP

Explain the cost of inserting at index 500 when you do not already have a node reference.

PRACTICE NOTES

An example, a trade-off, or a question to revisit.

03 / FOUNDATIONS

How does a hash table handle collisions?

Different keys may map to the same bucket. Separate chaining stores multiple entries per bucket; open addressing searches another slot. Lookup is expected $O(1)$ with a suitable hash function and controlled load factor, but can degrade. Keys still require equality checks after hashing. Resizing rehashes entries into a larger table.

FOLLOW-UP

What breaks if a mutable object changes its hash while used as a key?

04 / FOUNDATIONS

What is the difference between a process and a thread?

A process generally has an isolated virtual address space. Threads in one process share memory and resources but have separate stacks and execution state. Threads can communicate cheaply through shared memory, which introduces synchronization needs. A crash or race in one thread can affect the process. Actual scheduling and isolation depend on the operating system.

FOLLOW-UP

Why does a global variable need care when multiple threads write to it?

PRACTICE NOTES

An example, a trade-off, or a question to revisit.

05 / FOUNDATIONS

What is a race condition?

A race condition occurs when correctness depends on the timing of concurrent operations. Two workers can both read a counter as 10 and each write 11, losing an increment. Use an atomic operation, a lock, or a transactional database update. A lock must cover the complete read-modify-write sequence, not only the final write.

FOLLOW-UP

Design a concurrent stock decrement that cannot sell below zero.

06 / OBJECT-ORIENTED DESIGN

What is encapsulation?

Encapsulation keeps an object's representation behind a controlled interface so callers cannot violate its invariants. A bank account could expose deposit and withdraw methods instead of a writable balance field. Access modifiers alone are not enough if methods return mutable internal objects. The goal is a stable contract and valid state, not getters for every field.

FOLLOW-UP

How would you return a collection without allowing callers to mutate internal state?

PRACTICE NOTES

An example, a trade-off, or a question to revisit.

07 / DATABASES

How do primary and foreign keys differ?

A primary key uniquely identifies a row and is non-null. A foreign key constrains a reference to a candidate key in another or the same table. Together they help preserve relationships, but indexing of foreign-key columns depends on the database. Define deletion behavior deliberately, such as restricting deletes or cascading only when ownership is clear.

FOLLOW-UP

What should happen to order items when their parent order is deleted?

08 / DATABASES

What does a database index cost?

An index adds a searchable structure that can accelerate selected reads, joins, and ordering. It consumes storage and adds work to inserts, deletes, and updates of indexed values. A composite index's column order matters. Inspect a query plan and representative data before adding indexes; a low-selectivity predicate may still favor a table scan.

FOLLOW-UP

Would an index on a boolean column always make filtering faster?

PRACTICE NOTES

An example, a trade-off, or a question to revisit.

09 / DATABASES

What are ACID transactions?

Atomicity groups changes into all-or-nothing work. Consistency means enforced invariants remain valid. Isolation controls interference between concurrent transactions, with guarantees depending on the isolation level. Durability means committed changes survive failures within the database's durability contract. ACID does not automatically make external email or payment calls part of a database transaction.

FOLLOW-UP

Why should transferring money update both account balances in one transaction?

10 / WEB ESSENTIALS

How do HTTP methods express intent?

GET retrieves a representation and should be safe. POST commonly creates or triggers work. PUT replaces a resource at a known URI and is intended to be idempotent; PATCH applies a partial change whose idempotency depends on the operation. DELETE is intended to be idempotent in effect, although repeated responses may differ. Method names alone do not enforce application behavior.

FOLLOW-UP

Why is a payment retry with POST dangerous without an idempotency key?

PRACTICE NOTES

An example, a trade-off, or a question to revisit.

11 / WEB ESSENTIALS

What is the difference between authentication and authorization?

Authentication establishes an identity; authorization decides whether that identity may perform an action on a particular resource. A logged-in user still must not read another user's invoice. Check authorization on the server for every protected operation and use resource ownership or roles as appropriate. Hiding a button is only a user-interface decision.

FOLLOW-UP

Where would you check ownership in GET /orders/:id?

12 / WEB ESSENTIALS

How do you prevent SQL injection?

Use parameterized queries so data is bound separately from SQL syntax. Validate input shapes as well, but escaping or validation alone is not a substitute. Parameters usually cannot stand in for table names or sort directions; select those from a fixed allowlist. Give the database account only the permissions the application needs.

FOLLOW-UP

How would you safely implement user-selectable sorting?

PRACTICE NOTES

An example, a trade-off, or a question to revisit.

Your practice worksheet.

Choose one question from each section. Practice a 60-90 second explanation, then work through the follow-up. Record the specific gap you want to revisit.

Question / topic

My explanation in three sentences

A concrete example

The trade-off or edge case

What I will revisit next
